



ADVANCED SQL INTERVIEW HANDBOOK

SQL Joins & Set Operations

100+ Tricky Questions & Detailed
Solutions

100+

QUESTIONS

8

SECTIONS

Adv

LEVEL

*"Joins, NULLs, Sets, Anti/Semi-Joins & Edge Cases — explained the way
interviewers actually ask"*

CREATED BY

Kartik Mungole

@kartikmungole

Table of Contents

Your complete map to SQL Joins & Set Operations mastery

This guide covers **103 tricky, interview-style questions** on joins and set operations — from foundational NULL-handling quirks to advanced edge cases like cardinality explosion and collation mismatches.

Ref Sample Schema Reference · reference	03
S01 LEFT vs INNER JOIN & Filtering (WHERE vs ON Clause) · 15 questions	05
S02 FULL OUTER JOIN & Emulation Strategies · 12 questions	16
S03 CROSS JOIN & Cartesian Products · 12 questions	24
S04 SELF JOIN Patterns & Hierarchies · 16 questions	30
S05 Non-Equi Joins & Complex Predicates · 12 questions	43
S06 Set Operations: UNION vs UNION ALL · 12 questions	50
S07 Set Operations: EXCEPT/MINUS & INTERSECT · 12 questions	58
S08 Advanced Edge Cases & Set Logic Optimization · 12 questions	65

Sample Schema Reference

A common set of illustrative tables referenced throughout the examples in this guide (1 of 2)

Most examples in this guide use a small, consistent set of sample tables below. Some questions introduce ad-hoc tables (e.g. `products`, `students`, `web_logs`) purely to illustrate a single concept — these are described inline where used.

employees

Staff records, each linked to a department

Column	Type	Description
<code>id</code>	INT (PK)	Unique employee identifier
<code>name</code>	VARCHAR	Employee full name
<code>dept_id</code>	INT (FK)	References departments.id, may be NULL
<code>manager_id</code>	INT (FK)	Self-referencing FK to employees.id
<code>salary</code>	DECIMAL	Employee salary

departments

Company departments

Column	Type	Description
<code>id</code>	INT (PK)	Unique department identifier
<code>dept_name</code>	VARCHAR	Department name
<code>location</code>	VARCHAR	Office location / city

customers

Customer master data

Column	Type	Description
<code>id / cust_id</code>	INT (PK)	Unique customer identifier
<code>name</code>	VARCHAR	Customer name
<code>email</code>	VARCHAR	Customer contact email

Sample Schema Reference

A common set of illustrative tables referenced throughout the examples in this guide (2 of 2)

orders

Transactional order records

Column	Type	Description
id	INT (PK)	Unique order identifier
cust_id	INT (FK)	References customers.id
amount	DECIMAL	Order value
order_date	DATE	Date the order was placed

TableA / TableB

Generic example tables used for set-operation demos

Column	Type	Description
id	INT	Row identifier / join key
val / col	VARCHAR / NUMERIC	Sample value column used for comparisons

LEFT vs INNER JOIN & Filtering (WHERE vs ON Clause)

Tricky edge cases around predicate pushdown, NULL handling, and filtering timing.

Q001 What happens when you put a condition on the right table in the WHERE clause versus the ON clause of a LEFT JOIN?

- A In an **ON** clause, the filter is applied *before/during* the join. Unmatched left rows are still included (with NULLs for right columns). In a **WHERE** clause, the filter is applied *after* the join; since non-matching right columns are NULL, a condition like **WHERE right.col = 'X'** evaluates to **NULL = 'X'** (Unknown/False), turning the LEFT JOIN into an effective INNER JOIN.

```
SELECT e.name, d.dept_name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.id AND d.location
= 'NY'; -- Retains all employees

SELECT e.name, d.dept_name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.id
WHERE d.location = 'NY'; -- Filters out employees without
NY dept!
```

LEFT vs INNER JOIN & Filtering (WHERE vs ON Clause)

Tricky edge cases around predicate pushdown, NULL handling, and filtering timing.

Q002 How does an INNER JOIN handle NULL values in join keys?

A INNER JOIN uses equality comparison (`=`). In standard SQL, `NULL = NULL` evaluates to `UNKNOWN` (not TRUE). Therefore, rows where the join key is NULL in either table are filtered out and will never match.

```
SELECT * FROM TableA A INNER JOIN TableB B ON A.key =  
B.key; -- Rows with A.key IS NULL or B.key IS NULL are  
excluded.
```

LEFT vs INNER JOIN & Filtering (WHERE vs ON Clause)

Tricky edge cases around predicate pushdown, NULL handling, and filtering timing.

Q003 How do you perform a join on columns that may contain NULL values where NULL should match NULL?

A You can use the `IS NOT DISTINCT FROM` predicate (ANSI SQL standard) or `NULL-safe equal operator ( $\Leftrightarrow$ )` in MySQL/SQLite.

```
SELECT * FROM TableA A
JOIN TableB B ON A.key IS NOT DISTINCT FROM B.key;
-- In MySQL: ON A.key  $\Leftrightarrow$  B.key
```

LEFT vs INNER JOIN & Filtering (WHERE vs ON Clause)

Tricky edge cases around predicate pushdown, NULL handling, and filtering timing.

Q004 What is an Anti-Join and how is it implemented using a LEFT JOIN?

A An Anti-Join returns rows from the left table that have NO matching rows in the right table. It is implemented by doing a `LEFT JOIN` and adding `WHERE right_table.primary_key IS NULL`.

```
SELECT e.*
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.id
WHERE d.id IS NULL; -- Employees not assigned to any
department
```

LEFT vs INNER JOIN & Filtering (WHERE vs ON Clause)

Tricky edge cases around predicate pushdown, NULL handling, and filtering timing.

Q005 What happens if you use `WHERE right\_col IS NULL` vs `ON right\_col IS NULL` in a LEFT JOIN?

A `WHERE right_col IS NULL` creates an anti-join (selects unmatched rows). `ON right_col IS NULL` attempts to join right rows where that column is NULL, but still preserves all left rows regardless of whether a match was found.

```
SELECT * FROM A LEFT JOIN B ON A.id = B.a_id AND B.status IS NULL; -- Joins only B rows with NULL status, keeps all A rows.
```

Q006 What is the result of `SELECT \* FROM A LEFT JOIN B ON 1=1`?

A It acts as a CROSS JOIN (Cartesian product) where every row in A is paired with every row in B. If B is empty, all rows in A are still returned with NULLs for B's columns.

```
SELECT * FROM customers LEFT JOIN promotions ON 1=1;
```

LEFT vs INNER JOIN & Filtering (WHERE vs ON Clause)

Tricky edge cases around predicate pushdown, NULL handling, and filtering timing.

Q007 What happens in `SELECT \* FROM A LEFT JOIN B ON 1=0`?

A The join condition is always FALSE. No rows from B match, so the output contains all rows from A with NULLs for all B columns.

```
SELECT * FROM customers LEFT JOIN promotions ON 1=0;
```

Q008 Can an INNER JOIN produce more rows than the sum of rows in both tables?

A Yes! If there are duplicate join key values in both tables (e.g., 5 rows with key=1 in A, and 5 rows with key=1 in B), the result contains $5 \times 5 = 25$ rows (a Cartesian product per key group).

```
-- If TableA has 100 rows with key 'X' and TableB has 100  
rows with key 'X'  
-- INNER JOIN produces 10,000 rows!
```

LEFT vs INNER JOIN & Filtering (WHERE vs ON Clause)

Tricky edge cases around predicate pushdown, NULL handling, and filtering timing.

Q009 Does putting a condition on the LEFT table in the WHERE clause change LEFT JOIN behavior?

A Filtering the left table in the WHERE clause simply filters the left table before/after join; it does NOT convert the LEFT JOIN to an INNER JOIN (unlike filtering the right table). However, if the filter references right table columns, it will.

```
SELECT * FROM employees e LEFT JOIN departments d ON  
e.dept_id = d.id WHERE e.salary > 50000; -- Still a valid  
LEFT JOIN
```

LEFT vs INNER JOIN & Filtering (WHERE vs ON Clause)

Tricky edge cases around predicate pushdown, NULL handling, and filtering timing.

Q010 How do non-equi conditions in a LEFT JOIN ON clause behave?

A Non-equi conditions (like `>`, `<`, `BETWEEN`) match rows satisfying the range. Unmatched left rows are still preserved with NULLs for right table columns.

```
SELECT e.name, p.pay_grade
FROM employees e
LEFT JOIN salary_grades p ON e.salary BETWEEN p.min_sal
AND p.max_sal;
```

Q011 What is the difference between `LEFT JOIN` and `LEFT OUTER JOIN`?

A There is zero difference. `OUTER` is an optional keyword in SQL syntax. `LEFT JOIN` and `LEFT OUTER JOIN` are 100% identical.

```
SELECT * FROM A LEFT JOIN B ON A.id = B.id;
SELECT * FROM A LEFT OUTER JOIN B ON A.id = B.id; -- Exact
same query plan
```

LEFT vs INNER JOIN & Filtering (WHERE vs ON Clause)

Tricky edge cases around predicate pushdown, NULL handling, and filtering timing.

Q012 Why shouldn't you use `WHERE right\_table.col != 'value'` in a LEFT JOIN?

A Because for non-matching rows, `right_table.col` is NULL. Since `NULL ≠ 'value'` evaluates to UNKNOWN, those non-matching rows are discarded, inadvertently turning the query into an INNER JOIN and removing left-side rows!

```
-- WRONG (eliminates non-matching left rows):  
SELECT * FROM orders o LEFT JOIN discounts d ON o.id =  
d.order_id WHERE d.code ≠ 'SUMMER';  
  
-- CORRECT:  
SELECT * FROM orders o LEFT JOIN discounts d ON o.id =  
d.order_id AND d.code ≠ 'SUMMER';
```

LEFT vs INNER JOIN & Filtering (WHERE vs ON Clause)

Tricky edge cases around predicate pushdown, NULL handling, and filtering timing.

Q013 How to handle multiple join conditions with NULL check in LEFT JOIN?

A Combine join predicates inside the `ON` clause using `OR` or `COALESCE` depending on requirement.

```
SELECT * FROM users u
LEFT JOIN accounts a ON u.id = a.user_id OR (u.email =
a.email AND u.email IS NOT NULL);
```

Q014 What happens when joining on a expression that evaluates to NULL?

A If the expression yields NULL (e.g., `UPPER(column)` where column is NULL), the join condition evaluates to UNKNOWN and fails to match.

```
SELECT * FROM A JOIN B ON UPPER(A.code) = UPPER(B.code); -
- NULLs in code will not match.
```

LEFT vs INNER JOIN & Filtering (WHERE vs ON Clause)

Tricky edge cases around predicate pushdown, NULL handling, and filtering timing.

Q015 How does short-circuiting affect join conditions in SQL engines?

A SQL is declarative; query optimizers reorder conditions based on statistics. Never rely on left-to-right evaluation order in `ON` or `WHERE` clauses.

```
SELECT * FROM A JOIN B ON A.id = B.id AND  
complex_function(B.val) = 1; -- Engine decides order.
```

FULL OUTER JOIN & Emulation Strategies

Deep dive into FULL OUTER JOIN behavior, NULL handling, and emulation techniques for databases lacking native support.

Q016 How do you write a FULL OUTER JOIN in MySQL (which lacks native FULL OUTER JOIN support)?

A Combine a `LEFT JOIN` and a `RIGHT JOIN` (or `LEFT JOIN` with anti-join) using `UNION` (which automatically removes duplicate rows produced by both joins).

```
SELECT e.name, d.dept_name
FROM employees e LEFT JOIN departments d ON e.dept_id =
d.id
UNION
SELECT e.name, d.dept_name
FROM employees e RIGHT JOIN departments d ON e.dept_id =
d.id;
```

FULL OUTER JOIN & Emulation Strategies

Deep dive into FULL OUTER JOIN behavior, NULL handling, and emulation techniques for databases lacking native support.

Q017 Why is `UNION` used instead of `UNION ALL` when emulating FULL OUTER JOIN?

A `UNION` eliminates duplicate rows that are returned by both the LEFT JOIN and RIGHT JOIN (i.e., the matching rows). Using `UNION ALL` would duplicate all matched rows.

```
-- Using UNION ALL would return matching rows TWICE!  
-- Always use UNION unless you perform an explicit anti-join on the right side.
```

FULL OUTER JOIN & Emulation Strategies

Deep dive into FULL OUTER JOIN behavior, NULL handling, and emulation techniques for databases lacking native support.

Q018 How do you emulate FULL OUTER JOIN using 'UNION ALL' efficiently?

A Perform a `LEFT JOIN` for all left rows and matches, then `UNION ALL` with a `RIGHT JOIN` that explicitly filters for non-matching left rows (Anti-Join). This avoids the expensive distinct/sort operation of `'UNION'`.

```
SELECT e.name, d.dept_name
FROM employees e LEFT JOIN departments d ON e.dept_id =
d.id
UNION ALL
SELECT e.name, d.dept_name
FROM employees e RIGHT JOIN departments d ON e.dept_id =
d.id
WHERE e.dept_id IS NULL;
```

FULL OUTER JOIN & Emulation Strategies

Deep dive into FULL OUTER JOIN behavior, NULL handling, and emulation techniques for databases lacking native support.

Q019 What does a FULL OUTER JOIN return?

- A It returns all matching rows from both tables, plus all unmatched rows from the left table (with NULLs for right columns), plus all unmatched rows from the right table (with NULLs for left columns).

```
SELECT * FROM TableA A FULL OUTER JOIN TableB B ON A.id = B.id;
```

Q020 How can you isolate ONLY non-matching rows from both tables in a FULL OUTER JOIN?

- A Apply a WHERE clause checking that the primary key of either table IS NULL (where the other is present).

```
SELECT * FROM TableA A FULL OUTER JOIN TableB B ON A.id = B.id  
WHERE A.id IS NULL OR B.id IS NULL;
```

FULL OUTER JOIN & Emulation Strategies

Deep dive into FULL OUTER JOIN behavior, NULL handling, and emulation techniques for databases lacking native support.

Q021 How does `COALESCE` assist in FULL OUTER JOIN reporting?

A Since join key columns can be NULL from either side in unmatched rows, `COALESCE(A.key, B.key)` produces a single consolidated key column.

```
SELECT COALESCE(a.id, b.id) AS unified_id, a.val_a,  
       b.val_b  
FROM TableA a FULL OUTER JOIN TableB b ON a.id = b.id;
```

Q022 Can FULL OUTER JOIN produce duplicates if input tables have duplicate keys?

A Yes. If Table A has 2 rows with key 1, and Table B has 3 rows with key 1, FULL OUTER JOIN will produce 6 rows for key 1.

```
-- Duplicates multiply across matched keys, just like in  
INNER/LEFT JOIN.
```

FULL OUTER JOIN & Emulation Strategies

Deep dive into FULL OUTER JOIN behavior, NULL handling, and emulation techniques for databases lacking native support.

Q023 How does filtering in WHERE clause affect FULL OUTER JOIN?

A Filtering columns of both tables without NULL checks converts FULL OUTER JOIN into an INNER JOIN.

```
SELECT * FROM A FULL JOIN B ON A.id = B.id WHERE A.status = 'ACTIVE' AND B.status = 'ACTIVE'; -- Loses outer join property!
```

Q024 What is a FULL OUTER JOIN with non-equi condition?

A Matches rows satisfying range/condition; unmatched rows from both sides are preserved.

```
SELECT * FROM sales s FULL JOIN targets t ON s.amount BETWEEN t.min_amt AND t.max_amt;
```

FULL OUTER JOIN & Emulation Strategies

Deep dive into FULL OUTER JOIN behavior, NULL handling, and emulation techniques for databases lacking native support.

Q025 How to combine 3 tables with FULL OUTER JOIN?

A Chain FULL JOINS and use COALESCE for key preservation across chains.

```
SELECT COALESCE(a.id, b.id, c.id) AS id, a.val, b.val,
c.val
FROM A FULL JOIN B ON A.id = B.id
FULL JOIN C ON COALESCE(A.id, B.id) = C.id;
```

Q026 What is the memory footprint impact of FULL OUTER JOIN vs INNER JOIN?

A FULL OUTER JOIN cannot discard non-matching rows during stream execution; state must be kept to track unmatched keys from both sides, increasing memory requirement.

```
-- Query engine must maintain hash tables or sort inputs
completely.
```

FULL OUTER JOIN & Emulation Strategies

Deep dive into FULL OUTER JOIN behavior, NULL handling, and emulation techniques for databases lacking native support.

Q027 Is `RIGHT JOIN` just a syntactic reversal of `LEFT JOIN`?

A Yes. `A RIGHT JOIN B` is semantically identical to `B LEFT JOIN A`. Standardizing on LEFT JOIN is recommended for readability.

```
SELECT * FROM A RIGHT JOIN B ON A.id = B.id; -- Same as:  
SELECT * FROM B LEFT JOIN A ON A.id = B.id;
```

CROSS JOIN & Cartesian Products

Understanding cross joins, implicit syntax vs explicit syntax, matrix generation, and series generation.

Q028 How does a CROSS JOIN differ from an INNER JOIN with no ON condition?

A In standard SQL (ANSI SQL-92), explicit `CROSS JOIN` syntax does not accept an `ON` clause. `INNER JOIN` without an `ON` clause is either a syntax error in strict SQL modes or implicitly treated as a CROSS JOIN depending on RDBMS dialect.

```
-- Standard CROSS JOIN:
SELECT * FROM colors CROSS JOIN sizes;

-- INNER JOIN without ON clause (allowed in MySQL, syntax
error in PostgreSQL/SQL Server):'
SELECT * FROM colors INNER JOIN sizes;
```

CROSS JOIN & Cartesian Products

Understanding cross joins, implicit syntax vs explicit syntax, matrix generation, and series generation.

Q029 What is the row count result of a CROSS JOIN between Table A (M rows) and Table B (N rows)?

A The output will contain exactly $M \times N$ rows (the Cartesian product). If either table has 0 rows, the result has 0 rows.

```
-- Table A: 10 rows, Table B: 50 rows → CROSS JOIN output  
= 500 rows.
```

Q030 How can CROSS JOIN be useful in business reporting and calendar creation?

A It is used to generate complete matrix combinations, such as pairing every store with every date in a month to ensure zero-sales days appear in reports.

```
SELECT s.store_id, d.calendar_date  
FROM stores s  
CROSS JOIN dates_dim d  
WHERE d.year = 2026;
```

CROSS JOIN & Cartesian Products

Understanding cross joins, implicit syntax vs explicit syntax, matrix generation, and series generation.

Q031 What is the difference between comma-separated join `FROM A, B` and `A CROSS JOIN B`?

A They produce identical Cartesian products, but `CROSS JOIN` is explicit ANSI-SQL syntax, avoiding accidental missing join conditions.

```
SELECT * FROM A, B; -- Implicit cross join
SELECT * FROM A CROSS JOIN B; -- Explicit ANSI cross join
```

Q032 How to generate a sequence/series of numbers using CROSS JOIN?

A By cross joining small static CTEs/tables representing digits.

```
WITH Digits AS (SELECT 0 AS n UNION ALL SELECT 1 UNION ALL
SELECT 2 ...)
SELECT d1.n + d2.n * 10 + 1 AS seq
FROM Digits d1 CROSS JOIN Digits d2;
```

CROSS JOIN & Cartesian Products

Understanding cross joins, implicit syntax vs explicit syntax, matrix generation, and series generation.

Q033 What happens when you CROSS JOIN a table with an empty table?

A The result is 0 rows because $M \times 0 = 0$.

```
SELECT * FROM employees CROSS JOIN empty_table; -- Returns
0 rows
```

Q034 Can you add a WHERE clause to a CROSS JOIN?

A Yes! A CROSS JOIN followed by a WHERE clause is logically equivalent to an INNER JOIN with an ON clause.

```
SELECT * FROM A CROSS JOIN B WHERE A.id = B.id; --
Equivalent to A JOIN B ON A.id = B.id
```

Q035 How can CROSS JOIN cause severe database performance degradation?

A If large tables (e.g., 100K rows each) are cross joined unintentionally (missing join predicate), the output is 10 Billion rows, causing high CPU/Disk I/O or memory exhaustion.

```
-- Unintentional Cartesian Explosion!
```

CROSS JOIN & Cartesian Products

Understanding cross joins, implicit syntax vs explicit syntax, matrix generation, and series generation.

Q036 What is Lateral Cross Join / CROSS APPLY?

- A A specialized join where the right table expression (often a table-valued function or correlated subquery) is evaluated for each row of the left table.

```
SELECT u.id, p.*  
FROM users u  
CROSS JOIN LATERAL (SELECT * FROM posts p WHERE p.user_id  
= u.id ORDER BY created_at DESC LIMIT 3) p;
```

Q037 How to find missing combinations in data using CROSS JOIN and EXCEPT/LEFT JOIN?

- A Generate all possible pairs using CROSS JOIN, then subtract existing pairs.

```
SELECT s.id, p.id FROM stores s CROSS JOIN products p  
EXCEPT  
SELECT store_id, product_id FROM inventory;
```

CROSS JOIN & Cartesian Products

Understanding cross joins, implicit syntax vs explicit syntax, matrix generation, and series generation.

Q038 How does optimizer treat `CROSS JOIN` vs `INNER JOIN ON 1=1`?

A Most modern optimizers treat them identically in query execution plans.

```
EXPLAIN SELECT * FROM A CROSS JOIN B; -- Same plan as  
SELECT * FROM A JOIN B ON 1=1;
```

Q039 Can a CROSS JOIN be combined with LEFT JOIN in the same query?

A Yes. Operator precedence applies (joins are evaluated left to right unless parentheses are used).

```
SELECT * FROM A CROSS JOIN B LEFT JOIN C ON B.id = C.b_id;
```

SELF JOIN Patterns & Hierarchies

Mastering table self-referencing for hierarchical data, comparisons, consecutive records, and duplicate detection.

Q040 Write a query to find employees who earn more than their direct managers using a SELF JOIN.

A Self-join the employee table (`e`) with itself as manager (`m`) on `e.manager\_id = m.id` and filter `e.salary > m.salary`.

```
SELECT e.name AS employee_name, e.salary, m.name AS  
manager_name, m.salary AS manager_salary  
FROM employees e  
JOIN employees m ON e.manager_id = m.id  
WHERE e.salary > m.salary;
```

SELF JOIN Patterns & Hierarchies

Mastering table self-referencing for hierarchical data, comparisons, consecutive records, and duplicate detection.

Q041 How do you find all pairs of employees in the same department without returning duplicate or self pairs?

A Join the table to itself on `e1.dept\_id = e2.dept\_id` and use inequality `e1.id < e2.id` to prevent duplicate reciprocal pairs and self-matching.

```
SELECT e1.name AS emp1, e2.name AS emp2, e1.dept_id
FROM employees e1
JOIN employees e2 ON e1.dept_id = e2.dept_id AND e1.id <
e2.id;
```

SELF JOIN Patterns & Hierarchies

Mastering table self-referencing for hierarchical data, comparisons, consecutive records, and duplicate detection.

Q042 How do you find consecutive log entries/events using a SELF JOIN?

A Join table `t1` with `t2` on date difference condition (e.g., `t2.date = t1.date + 1 day`) to compare adjacent records.

```
SELECT t1.user_id, t1.login_date AS day1, t2.login_date AS day2
FROM user_logins t1
JOIN user_logins t2 ON t1.user_id = t2.user_id AND
t2.login_date = t1.login_date + INTERVAL '1 day';
```

SELF JOIN Patterns & Hierarchies

Mastering table self-referencing for hierarchical data, comparisons, consecutive records, and duplicate detection.

Q043 How to list all employees and their managers, including top executives who have no manager?

A Use a `LEFT JOIN` on self table so employees with `manager\_id` IS NULL` are retained in output.

```
SELECT e.name AS employee, COALESCE(m.name, 'Top Boss') AS  
manager  
FROM employees e  
LEFT JOIN employees m ON e.manager_id = m.id;
```

SELF JOIN Patterns & Hierarchies

Mastering table self-referencing for hierarchical data, comparisons, consecutive records, and duplicate detection.

Q044 How to detect overlapping time intervals in the same table using a SELF JOIN?

A Join on `a.id != b.id` and condition `a.start\_time < b.end\_time AND a.end\_time > b.start\_time`.

```
SELECT a.id AS booking1, b.id AS booking2
FROM bookings a
JOIN bookings b ON a.room_id = b.room_id AND a.id < b.id
WHERE a.start_date < b.end_date AND a.end_date >
b.start_date;
```

SELF JOIN Patterns & Hierarchies

Mastering table self-referencing for hierarchical data, comparisons, consecutive records, and duplicate detection.

Q045 How do you identify duplicate email addresses and keep only the record with the minimum ID using SELF JOIN?

A Join table `t1` and `t2` on email match where `t1.id > t2.id`.

```
SELECT DISTINCT t1.id, t1.email
FROM users t1
JOIN users t2 ON t1.email = t2.email AND t1.id > t2.id;
```

Q046 How do you traverse multi-level organizational hierarchy (Grandmanager) using SELF JOIN?

A Chain two self joins: join employee to manager, and manager to grand-manager.

```
SELECT e.name AS emp, m.name AS manager, gm.name AS
grand_manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.id
LEFT JOIN employees gm ON m.manager_id = gm.id;
```

SELF JOIN Patterns & Hierarchies

Mastering table self-referencing for hierarchical data, comparisons, consecutive records, and duplicate detection.

Q047 Find customers who bought Product A and Product B on different days using SELF JOIN.

A Self-join purchases table on customer_id where product_id='A' on side 1 and product_id='B' on side 2 and dates differ.

```
SELECT DISTINCT p1.customer_id
FROM purchases p1
JOIN purchases p2 ON p1.customer_id = p2.customer_id
WHERE p1.product_id = 'A' AND p2.product_id = 'B' AND
p1.purchase_date ≠ p2.purchase_date;
```

SELF JOIN Patterns & Hierarchies

Mastering table self-referencing for hierarchical data, comparisons, consecutive records, and duplicate detection.

Q048 How do you calculate running totals or prior period changes without window functions using SELF JOIN?

A Join table `t1` to `t2` where `t2.date <= t1.date` and group by `t1.date`.

```
SELECT t1.trans_date, t1.amount, SUM(t2.amount) AS  
cumulative_amount  
FROM daily_sales t1  
JOIN daily_sales t2 ON t2.trans_date ≤ t1.trans_date  
GROUP BY t1.trans_date, t1.amount;
```

SELF JOIN Patterns & Hierarchies

Mastering table self-referencing for hierarchical data, comparisons, consecutive records, and duplicate detection.

Q049 How do you find gaps in sequential ID numbers using SELF JOIN?

A Join `t1` with `t2` on `t2.id = t1.id + 1` with a `LEFT JOIN` where `t2.id IS NULL`.

```
SELECT t1.id + 1 AS missing_start
FROM sequence_table t1
LEFT JOIN sequence_table t2 ON t1.id + 1 = t2.id
WHERE t2.id IS NULL AND t1.id < (SELECT MAX(id) FROM
sequence_table);
```

Q050 What is the performance drawback of multi-level SELF JOINS compared to Recursive CTEs?

A Multi-level self joins require fixed tree depth and scale poorly with deep hierarchies. Recursive CTEs dynamically traverse arbitrary depths efficiently.

```
-- Self joins require N explicit joins for N levels depth!
```

SELF JOIN Patterns & Hierarchies

Mastering table self-referencing for hierarchical data, comparisons, consecutive records, and duplicate detection.

Q051 How to find nodes in a graph that have a 2-hop loop (A → B → A)?

A Self join graph edges on target=source and source=target.

```
SELECT e1.from_node, e1.to_node
FROM edges e1
JOIN edges e2 ON e1.to_node = e2.from_node AND e2.to_node
= e1.from_node
WHERE e1.from_node < e1.to_node;
```

SELF JOIN Patterns & Hierarchies

Mastering table self-referencing for hierarchical data, comparisons, consecutive records, and duplicate detection.

Q052 How to compute price change percentage relative to previous record using SELF JOIN?

A Join each row with the row having the immediate preceding timestamp.

```
SELECT curr.id, curr.price, prev.price AS prev_price,  
       ((curr.price - prev.price)/prev.price)*100 AS pct_change  
FROM prices curr  
JOIN prices prev ON curr.id = prev.id + 1;
```

Q053 Find all students sharing both same grade AND section.

A Self join on grade and section with `s1.student\_id < s2.student\_id`.

```
SELECT s1.name, s2.name, s1.grade, s1.section  
FROM students s1 JOIN students s2 ON s1.grade = s2.grade  
AND s1.section = s2.section AND s1.id < s2.id;
```

SELF JOIN Patterns & Hierarchies

Mastering table self-referencing for hierarchical data, comparisons, consecutive records, and duplicate detection.

Q054 How to check if a table is symmetric (if (A,B) exists then (B,A) exists)?

A Self join and count matches vs total distinct pairs.

```
SELECT f1.x, f1.y FROM Functions f1 JOIN Functions f2 ON
f1.x = f2.y AND f1.y = f2.x WHERE f1.x ≤ f1.y GROUP BY
f1.x, f1.y HAVING COUNT(*) > 1 OR f1.x ≠ f1.y;
```

SELF JOIN Patterns & Hierarchies

Mastering table self-referencing for hierarchical data, comparisons, consecutive records, and duplicate detection.

Q055 Find users who logged in 3 or more consecutive days using SELF JOIN.

A Self join log table twice for `date + 1` and `date + 2`.

```
SELECT DISTINCT l1.user_id
FROM logins l1
JOIN logins l2 ON l1.user_id = l2.user_id AND
l2.login_date = l1.login_date + INTERVAL '1 day'
JOIN logins l3 ON l1.user_id = l3.user_id AND
l3.login_date = l1.login_date + INTERVAL '2 day';
```

Non-Equi Joins & Complex Predicates

Joins utilizing inequalities (<, >, BETWEEN, LIKE, overlapping ranges, conditional logic).

Q056 What is a Non-Equi Join? Provide a real-world scenario.

A A Non-Equi Join uses join operators other than equality (`=`), such as `<`, `>`, `<=` , `>=` , `BETWEEN`, or `!=`. Useful for matching salaries into grade brackets.

```
SELECT e.name, e.salary, g.grade_level
FROM employees e
JOIN salary_grades g ON e.salary BETWEEN g.min_salary AND
g.max_salary;
```

Q057 How do you perform fuzzy string matching in a join?

A Use the `LIKE` operator or `ILIKE` / string matching functions inside the `ON` clause.

```
SELECT * FROM product_catalog p
JOIN search_terms s ON p.title LIKE CONCAT('%', s.keyword,
'%');
```

Non-Equi Joins & Complex Predicates

Joins utilizing inequalities (<, >, BETWEEN, LIKE, overlapping ranges, conditional logic).

Q058 How do you join effective-dated history tables to find active dimension values on transaction dates?

A Join on surrogate key and test transaction date between `effective\_start\_date` and `effective\_end\_date`.

```
SELECT t.trans_id, t.amount, d.customer_name
FROM transactions t
JOIN customer_dim_history d ON t.customer_id =
d.customer_id
    AND t.trans_date ≥ d.start_date AND t.trans_date <
d.end_date;
```

Non-Equi Joins & Complex Predicates

Joins utilizing inequalities (<, >, BETWEEN, LIKE, overlapping ranges, conditional logic).

Q059 How to calculate rolling aggregates using Non-Equi Join?

A Join table `a` with `b` on `b.date` between `a.date - 7 days` and `a.date`.

```
SELECT a.date, SUM(b.amount) AS rolling_7d_total
FROM daily_sales a
JOIN daily_sales b ON b.date BETWEEN a.date - INTERVAL '6
day' AND a.date
GROUP BY a.date;
```

Q060 What is the risk of performance degradation with Non-Equi Joins?

A Hash joins cannot be utilized for non-equi joins (e.g. `!=` or `<`). The engine falls back to Nested Loop joins, which can lead to $O(N * M)$ performance complexity.

```
-- Optimizers struggle to index non-equi predicate joins
without spatial/range indexes.
```

Non-Equi Joins & Complex Predicates

Joins utilizing inequalities (<, >, BETWEEN, LIKE, overlapping ranges, conditional logic).

Q061 How to do a conditional join where criteria differ based on row type?

A Use `CASE` statement inside the `ON` clause or `OR` predicates.

```
SELECT * FROM orders o
JOIN entity e ON (o.type = 'INDIVIDUAL' AND o.user_id =
e.id)
                OR (o.type = 'CORP' AND o.company_id = e.id);
```

Q062 How do you join tables on geographical bounding boxes / range conditions?

A Join using range inequality conditions on latitude and longitude.

```
SELECT p.place_name, z.zone_name
FROM points p
JOIN zones z ON p.lat BETWEEN z.min_lat AND z.max_lat AND
p.lon BETWEEN z.min_lon AND z.max_lon;
```

Non-Equi Joins & Complex Predicates

Joins utilizing inequalities (<, >, BETWEEN, LIKE, overlapping ranges, conditional logic).

Q063 How to join on JSON array contents or string lists?

A Use array containment operators or string position functions in the ON clause.

```
SELECT * FROM tags t
JOIN posts p ON POSITION(t.tag_name IN p.tag_list) > 0;
```

Q064 Can you use CTE result inside a Non-Equi Join?

A Yes, CTEs behave just like tables/views in join clauses.

```
WITH RangeTable AS (SELECT 1 AS low, 100 AS high)
SELECT * FROM data d JOIN RangeTable r ON d.val BETWEEN
r.low AND r.high;
```

Non-Equi Joins & Complex Predicates

Joins utilizing inequalities (<, >, BETWEEN, LIKE, overlapping ranges, conditional logic).

Q065 How to map numeric score thresholds to letter grades using non-equi join?

A Join student scores to grading scale table on score range.

```
SELECT s.student_name, s.score, g.letter_grade
FROM students s
JOIN grading_scale g ON s.score ≥ g.min_score AND s.score
≤ g.max_score;
```

Q066 How to join two tables when matching keys require regex patterns?

A Use regex matching operators (e.g., `~` in Postgres or `REGEXP` in MySQL) in `ON` clause.

```
SELECT * FROM web_logs l JOIN url_rules r ON l.url ~
r.pattern_regex;
```

Non-Equi Joins & Complex Predicates

Joins utilizing inequalities (<, >, BETWEEN, LIKE, overlapping ranges, conditional logic).

Q067 How do you find inventory availability by joining demand and supply dates?

A Join supply events and demand events where supply_date <= demand_date.

```
SELECT d.order_id, d.demand_date, s.supply_id,  
       s.supply_date  
FROM demand d  
JOIN supply s ON d.item_id = s.item_id AND s.supply_date  
              ≤ d.demand_date;
```

Set Operations: UNION vs UNION ALL

Logic, performance differences, NULL handling, data type matching, and implicit sorting.

Q068 What is the key functional and operational difference between UNION and UNION ALL?

A **UNION** combines datasets and performs a distinct operation to remove duplicate rows, requiring an implicit sort/hash operation.

UNION ALL combines datasets retaining all rows including duplicates, making it substantially faster and memory-efficient.

```
SELECT col FROM TableA
UNION -- Deduplicates result set
SELECT col FROM TableB;
```

```
SELECT col FROM TableA
UNION ALL -- Preserves duplicates, much faster!
SELECT col FROM TableB;
```

Set Operations: UNION vs UNION ALL

Logic, performance differences, NULL handling, data type matching, and implicit sorting.

Q069 What are the strict column matching requirements for set operations (UNION/EXCEPT/INTERSECT)?

- A
1. Both queries must return the exact same number of columns.
 2. Corresponding columns must have compatible data types.
 3. Order of columns in SELECT list must align.

```
SELECT id, name FROM A UNION SELECT id, name FROM B; --  
Valid  
SELECT id, name FROM A UNION SELECT name, id FROM B; --  
Error or type mismatch!
```

Set Operations: UNION vs UNION ALL

Logic, performance differences, NULL handling, data type matching, and implicit sorting.

Q070 How are column names determined in the result set of a UNION?

A The output column names are always inherited from the **first** query in the UNION sequence.

```
SELECT emp_id AS employee_code FROM employees
UNION ALL
SELECT mgr_id FROM managers; -- Output column header will
be 'employee_code'
```

Q071 Where should ORDER BY and LIMIT clauses be placed in a UNION query?

A They must be placed at the very end of the final query to sort/limit the combined result set. Individual queries cannot have ORDER BY unless wrapped in subqueries.

```
SELECT name FROM clients
UNION ALL
SELECT name FROM vendors
ORDER BY name ASC
LIMIT 10;
```

Set Operations: UNION vs UNION ALL

Logic, performance differences, NULL handling, data type matching, and implicit sorting.

Q072 How does `UNION` treat NULL values across query results?

A In set operations, `NULL` is treated as equal to `NULL`. Multiple `NULL` values across combined sets will be deduplicated into a single `NULL` row when using `UNION`.

```
SELECT NULL UNION SELECT NULL; -- Returns 1 row containing NULL
```

Q073 What happens if you combine `UNION` and `UNION ALL` in a single SQL statement?

A Set operations are evaluated from left to right unless parentheses are used. Operators with higher sequence run first.

```
SELECT col FROM A UNION SELECT col FROM B UNION ALL SELECT col FROM C;  
-- B & A deduplicated first, then C appended with duplicates.
```

Set Operations: UNION vs UNION ALL

Logic, performance differences, NULL handling, data type matching, and implicit sorting.

Q074 Can you perform aggregation (`GROUP BY`) on the output of a `UNION ALL`?

A Yes, wrap the `UNION ALL` inside a subquery or CTE and aggregate the combined set.

```
WITH Combined AS (  
    SELECT category, sales FROM store_east  
    UNION ALL  
    SELECT category, sales FROM store_west  
)  
SELECT category, SUM(sales) FROM Combined GROUP BY  
category;
```

Set Operations: UNION vs UNION ALL

Logic, performance differences, NULL handling, data type matching, and implicit sorting.

Q075 Why is `UNION ALL` preferred in Data Warehousing (ETL/ELT) pipelines?

A Because ETL processes usually extract distinct partitions (e.g., historical vs active partition). Deduplication is redundant and `UNION ALL` avoids expensive CPU/RAM sort operations.

```
-- Pipeline efficiency optimization standard.
```

Q076 How to handle mismatched column types in a UNION (e.g. INT in Query 1, VARCHAR in Query 2)?

A Explicitly CAST/CONVERT data types so corresponding columns match.

```
SELECT CAST(user_id AS VARCHAR) AS id, username FROM users
UNION ALL
SELECT legacy_code AS id, company_name FROM
legacy_clients;
```

Set Operations: UNION vs UNION ALL

Logic, performance differences, NULL handling, data type matching, and implicit sorting.

Q077 What happens when Query 1 has 3 columns and Query 2 has 2 columns?

A SQL engine raises a syntax error: 'Each UNION query must have an equal number of expression lists'. Fix by padding NULL constants.

```
SELECT id, name, salary FROM staff
UNION ALL
SELECT id, name, NULL AS salary FROM contractors;
```

Q078 How does `UNION` behavior differ from `JOIN`?

A Joins combine columns vertically/horizontally across tables based on keys (widening the table). Set operations (UNION) stack rows vertically (lengthening the table).

```
-- JOIN = adds attributes (columns); UNION = adds
instances (rows).
```

Set Operations: UNION vs UNION ALL

Logic, performance differences, NULL handling, data type matching, and implicit sorting.

Q079 How to tag source table origin in a combined UNION query?

A Add a literal string/constant column to each SELECT list.

```
SELECT id, name, 'Employee' AS record_source FROM
employees
UNION ALL
SELECT id, name, 'Contractor' AS record_source FROM
contractors;
```

Set Operations: EXCEPT/MINUS & INTERSECT

Set difference and intersection logic, dialect differences, and emulation using joins.

Q080 What does the `EXCEPT` (or `MINUS`) set operator do, and how does it handle duplicates?

A `EXCEPT` (called `MINUS` in Oracle) returns distinct rows from the left query that do NOT exist in the right query result. Duplicate rows in the left query are deduplicated prior to subtraction.

```
SELECT product_id FROM catalog
EXCEPT -- (or MINUS)
SELECT product_id FROM ordered_items; -- Catalog items
never ordered
```

Set Operations: EXCEPT/MINUS & INTERSECT

Set difference and intersection logic, dialect differences, and emulation using joins.

Q081 How do you emulate `EXCEPT` / `MINUS` in MySQL (which does not natively support EXCEPT in older versions)?

A Using a `LEFT JOIN` filtering for NULLs, or using `NOT EXISTS` / `NOT IN`.

```
SELECT c.product_id
FROM catalog c
LEFT JOIN ordered_items o ON c.product_id = o.product_id
WHERE o.product_id IS NULL;
```

Set Operations: EXCEPT/MINUS & INTERSECT

Set difference and intersection logic, dialect differences, and emulation using joins.

Q082 What does `INTERSECT` do, and how does it treat NULLs?

A `INTERSECT` returns only distinct rows that are present in BOTH query results. Like `UNION` and `EXCEPT`, `INTERSECT` treats two NULL values as equal.

```
SELECT customer_id FROM online_orders
INTERSECT
SELECT customer_id FROM store_orders; -- Customers who
bought both online and in-store
```

Q083 How do you emulate `INTERSECT` using JOIN?

A Use `INNER JOIN` with `SELECT DISTINCT` across both datasets on all matching columns.

```
SELECT DISTINCT o.customer_id
FROM online_orders o
INNER JOIN store_orders s ON o.customer_id =
s.customer_id;
```

Set Operations: EXCEPT/MINUS & INTERSECT

Set difference and intersection logic, dialect differences, and emulation using joins.

Q084 What is the difference between `EXCEPT` and `NOT IN` regarding NULLs?

A If the subquery in a `NOT IN` returns even a single `NULL`, the entire `NOT IN` query returns ZERO rows! `EXCEPT` safely treats NULLs as distinct values without failing.

```
-- DANGER with NOT IN:  
SELECT id FROM A WHERE id NOT IN (SELECT id FROM B); --  
Fails/empty if B contains NULL!  
  
-- SAFE:  
SELECT id FROM A EXCEPT SELECT id FROM B;
```

Set Operations: EXCEPT/MINUS & INTERSECT

Set difference and intersection logic, dialect differences, and emulation using joins.

Q085 What is `EXCEPT ALL` and how does it differ from standard `EXCEPT`?

A Standard `EXCEPT` deduplicates rows before returning difference. `EXCEPT ALL` considers row occurrence counts (if row appears 3 times in A and 1 time in B, `EXCEPT ALL` returns 2 occurrences in output).

```
-- SQL Standard feature supported in PostgreSQL/DB2.
```

Q086 What is `INTERSECT ALL`?

A It returns intersection taking row multiplicity into account (min count of occurrences between set A and set B).

```
-- Supported in SQL Standard / PostgreSQL.
```

Set Operations: EXCEPT/MINUS & INTERSECT

Set difference and intersection logic, dialect differences, and emulation using joins.

Q087 How do set operator precedence rules apply when combining UNION, INTERSECT, and EXCEPT?

- A `INTERSECT` has higher precedence than `UNION` and `EXCEPT`. Operations run left-to-right among operators of equal precedence unless overridden by parentheses.

```
SELECT col FROM A UNION SELECT col FROM B INTERSECT SELECT  
col FROM C;  
-- Evaluated as: A UNION (B INTERSECT C)
```

Q088 Can you use `EXCEPT` on multi-column SELECT lists?

- A Yes, as long as number and types of columns match. The set difference is evaluated across the entire tuple/row.

```
SELECT first_name, last_name, dob FROM candidates  
EXCEPT  
SELECT first_name, last_name, dob FROM employees;
```

Set Operations: EXCEPT/MINUS & INTERSECT

Set difference and intersection logic, dialect differences, and emulation using joins.

Q089 How to find records present in Table A or Table B, but NOT in both (Symmetric Difference)?

A Combine (A EXCEPT B) UNION (B EXCEPT A), or use FULL OUTER JOIN with anti-condition.

```
(SELECT id FROM A EXCEPT SELECT id FROM B)
UNION
(SELECT id FROM B EXCEPT SELECT id FROM A);
```

Q090 How does `EXCEPT` handle all-NULL rows?

A If both sets contain a row with all NULL columns, `EXCEPT` will subtract it, returning no row for that entry.

```
SELECT NULL, NULL EXCEPT SELECT NULL, NULL; -- Returns 0
rows
```

Q091 Which engine operator is typically used under the hood for `INTERSECT`?

A Hash Match (Right Semi Join) or Intersect Hash operator / Sort Merge Intersect.

```
EXPLAIN SELECT id FROM A INTERSECT SELECT id FROM B;
```

Advanced Edge Cases & Set Logic Optimization

Semi-joins, Anti-joins, Cardinality explosion, dynamic multi-table joins, and tricky interview challenges.

Q092 What is the difference between an INNER JOIN and a Semi-Join (`IN` / `EXISTS`)?

A An `INNER JOIN` returns combined columns from both tables and duplicates left rows if there are multiple matches in the right table. A `Semi-Join` evaluates existence only and guarantees left table rows are returned AT MOST ONCE, regardless of duplicate matches.

```
-- Semi-join (no row multiplication):  
SELECT * FROM department d WHERE EXISTS (SELECT 1 FROM  
employee e WHERE e.dept_id = d.id);
```

Advanced Edge Cases & Set Logic Optimization

Semi-joins, Anti-joins, Cardinality explosion, dynamic multi-table joins, and tricky interview challenges.

Q093 Why is `EXISTS` generally safer than `IN` for correlated subqueries?

- A
1. `EXISTS` stops evaluation as soon as the first match is found (short-circuit).
 2. `EXISTS` is immune to NULL value traps that break `NOT IN` logic.

```
SELECT * FROM customers c WHERE EXISTS (SELECT 1 FROM
orders o WHERE o.cust_id = c.id);
```

Advanced Edge Cases & Set Logic Optimization

Semi-joins, Anti-joins, Cardinality explosion, dynamic multi-table joins, and tricky interview challenges.

Q094 What is 'Cardinality Explosion' in SQL joins and how do you fix or prevent it?

A It occurs when joining tables on non-unique keys (many-to-many relationship), resulting in a massive multiplication of rows (e.g. 10K rows x 10K rows = 100 Million rows), causing memory crashes and wrong SUM aggregations. Fixed by pre-aggregating or deduplicating before joining.

```
-- Pre-aggregate before join to preserve grain:
WITH AggOrders AS (
    SELECT cust_id, SUM(amount) AS total_amt FROM orders
    GROUP BY cust_id
)
SELECT c.cust_id, c.name, o.total_amt
FROM customers c JOIN AggOrders o ON c.cust_id =
o.cust_id;
```

Advanced Edge Cases & Set Logic Optimization

Semi-joins, Anti-joins, Cardinality explosion, dynamic multi-table joins, and tricky interview challenges.

Q095 How does joining aggregated CTEs differ from joining raw tables and grouping afterward?

A Pre-aggregating in CTEs maintains distinct grain (1 row per key), preventing join fan-out and double-counting errors in sums/counts across multiple 1-to-many relationships.

```
-- Avoid Joining raw orders AND raw payments in one query!  
Aggregate each in separate CTEs first.
```

Advanced Edge Cases & Set Logic Optimization

Semi-joins, Anti-joins, Cardinality explosion, dynamic multi-table joins, and tricky interview challenges.

Q096 How do you perform a Join on dynamic ranges using CTEs and `BETWEEN`?

A Construct range CTEs (e.g., date buckets or pricing tiers) and perform non-equi join.

```
WITH Tiers AS (  
    SELECT 'Low' AS tier, 0 AS min_val, 100 AS max_val  
    UNION ALL  
    SELECT 'High', 101, 1000  
)  
SELECT d.item, t.tier FROM data d JOIN Tiers t ON d.val  
BETWEEN t.min_val AND t.max_val;
```

Advanced Edge Cases & Set Logic Optimization

Semi-joins, Anti-joins, Cardinality explosion, dynamic multi-table joins, and tricky interview challenges.

Q097 Write a query to compare two tables and return 'MATCH' if identical or list the differences.

A Use `(A EXCEPT B) UNION ALL (B EXCEPT A)` inside a CTE. If CTE count is 0, tables are identical.

```
WITH Diff AS (  
    (SELECT id, val, 'In A only' AS src FROM TableA EXCEPT  
     SELECT id, val, 'In A only' FROM TableB)  
    UNION ALL  
    (SELECT id, val, 'In B only' AS src FROM TableB EXCEPT  
     SELECT id, val, 'In B only' FROM TableA)  
)  
SELECT * FROM Diff;
```

Advanced Edge Cases & Set Logic Optimization

Semi-joins, Anti-joins, Cardinality explosion, dynamic multi-table joins, and tricky interview challenges.

Q098 What happens when joining two tables on floating-point decimal numbers?

A Floating-point numbers (`FLOAT`/`REAL`) suffer from precision representation issues where ``0.1 + 0.2 != 0.3``. Join conditions on exact floating equality can silently fail! Always use ``NUMERIC`/`DECIMAL`` or rounding.

```
SELECT * FROM A JOIN B ON ROUND(A.float_col, 4) =  
ROUND(B.float_col, 4);
```

Q099 How to perform a Full Outer Anti-Join?

A Perform a FULL OUTER JOIN and filter where either side's join key is NULL.

```
SELECT * FROM TableA a FULL OUTER JOIN TableB b ON a.id =  
b.id  
WHERE a.id IS NULL OR b.id IS NULL;
```

Advanced Edge Cases & Set Logic Optimization

Semi-joins, Anti-joins, Cardinality explosion, dynamic multi-table joins, and tricky interview challenges.

Q100 How does collation affect string comparison in JOINS?

A If joined columns have differing collations (e.g., `utf8mb4\_unicode\_ci` vs `latin1\_swedish\_ci`), the database raises a 'collation mismatch' error. Explicit collation conversion is required.

```
SELECT * FROM A JOIN B ON A.str_col COLLATE  
utf8mb4_unicode_ci = B.str_col COLLATE utf8mb4_unicode_ci;
```

Q101 Explain the 'Chained Join' performance trap in multi-table queries.

A Joining 5+ tables without filtering early causes intermediate result set inflation. Optimizers rely on predicate pushdown; missing indexes on foreign keys force multiple full table scans.

```
-- Standard best practice: Filter early using  
subqueries/CTEs before multi-table joins.
```

Advanced Edge Cases & Set Logic Optimization

Semi-joins, Anti-joins, Cardinality explosion, dynamic multi-table joins, and tricky interview challenges.

Q102 How to combine data from 2 tables with different schemas into a single normalized output using Set Operations?

A Select common target columns from each table, mapping missing fields to NULL or default values.

```
SELECT user_id AS entity_id, email, 'User' AS entity_type,
created_at FROM users
UNION ALL
SELECT comp_id AS entity_id, contact_email AS email,
'Company' AS entity_type, registration_date AS created_at
FROM companies;
```

Advanced Edge Cases & Set Logic Optimization

Semi-joins, Anti-joins, Cardinality explosion, dynamic multi-table joins, and tricky interview challenges.

Q103 How do window functions interact with JOIN clauses during query execution order?

A In SQL order of execution, `FROM` and `JOIN` happen first, followed by `WHERE`, `GROUP BY`, `HAVING`, and THEN `WINDOW FUNCTIONS`. Thus, window functions evaluate on the joined result set.

```
-- Execution Order: FROM/JOIN → WHERE → GROUP BY →  
HAVING → WINDOW FUNCTIONS → SELECT → DISTINCT → ORDER  
BY → LIMIT
```